

## Introduction:

The aim of this assignment was to apply two neural computing methods to a dataset and critically compare their results. The dataset chosen, 'Default of Credit Card Clients', was sourced from UCI Machine Learning Repository (Yeh and Lien, 2009), containing 30,000 instances and 23 different features. The aim of the models was to predict the default payment of credit card customers, making this a binary classification task, seeking to classify instances of non-default (0) and default (1). The initial exploratory data analysis uncovered a substantial class imbalance, with 78% of occurrences being 'non-default'. This imbalance proved to be the most challenging aspect of the project, as both of my methodologies struggled to correctly predict the minority class.

The first method, the Multilayer Perceptron, was selected due to its universal approximation capabilities (Hornik, 1991), making it well suited for non-linear data, as is the case of credit card information. In addition, this method's flexibility and versatility in handling different features after standardisation, further supported its selection as a top contender. The second method chosen, Support Vector Machines, is highly efficient in high dimensional spaces, thus proving ideal for a 23-feature dataset. The RBF kernel's ability to capture non-linearity rendered SVMs as particularly suitable for the dataset, given the increased likelihood of non-linear class boundaries in financial data (Huang *et al.*, 2007).

Before beginning the project, I hypothesised that the MLP, with regularisation and momentum, would perform better than SVMs on this credit card dataset. This was based on the MLP's combination of flexibility within a non-linear environment; therefore, this method would be able to better adapt to the dataset, while adjusting to outliers and noisy data.

I began this project by preprocessing the data. While no missing values were found, there were multiple undocumented values in the columns 'EDUCATION' and 'MARRIAGE'. They were thus reassigned onto the existing category 'other', to ensure all data was correctly classified. The data was then stratified and a train/validation/test split was applied, with the validation set being used for hyperparameter tuning. Given the substantial size of the dataset, feature scaling was deemed necessary. In order to prevent data leakage, the scaler was fitted exclusively on the training set before being applied to all three splits. Standardisation was chosen over min-max scaling due to the latter's sensitivity to outliers, which are often present in financial datasets.

Before any class weights were computed, a baseline model was first constructed by utilising BCELoss(), which would not have any class weights. This first MLP model was built in PyTorch, comprising of two hidden layers of 64 neurons each, with ReLU activation function and a Sigmoid function was applied in the output layer. The loss curve results are shown in Figure 1. The baseline model demonstrated clear overfitting after epoch 20, justifying the introduction of early stopping, regularisation techniques and class weights in the final models.

When looking at metrics, ROC-AUC and the F1 score were chosen over accuracy due to the substantial class imbalance, which rendered the accuracy score misleading; a naïve classifier simply predicting non-default would achieve 78% accuracy. ROC-AUC measures the model's ability to distinguish and classify correctly the classes across all thresholds; while the F1 score, which balances precision and recall, is especially important in financial data where both false positives and false negatives carry real consequences.



Figure 1

## Methodology:

The final MLP model utilises a funnel architecture (23-128-64-32-1). The input layer takes all 23 features, and three hidden layers which progressively reduce dimensionality as the network learns the compressed representations. The output is a single neuron, ideal for binary classification. The ReLU activation function was selected for the hidden layers, as it avoids the vanishing gradient problem, allowing deeper networks to train accurately. The dropout rate of 0.3 prevents the co-adaptation of neurons and it helps reduce overfitting (Srivastava et al., 2014), while L2 regularisation through the use of weight decay improves generalisation.

The introduction of BCEWithLogitsLoss with pos\_weight=2.0 combines Sigmoid with BCE into one numerically stable function. The initial pos\_weight (3.52) had been calculated as the ratio of non-default to default cases. This was thus reduced to 2.0 in the optimised model to avoid overcorrection. The Adam optimiser was applied for easier adaptation of learning rate per parameter and its automatic incorporation of momentum (Kingma and Ba, 2015). Finally, early stopping with a patience of 10 was incorporated, to avoid unnecessary computation and to save the best model each time validation loss improved and stopping training once no improvement was observed.

A grid search was also implemented with learning rates 0.001, 0.01 and 0.1, and dropout 0.3 and 0.5. The results of the grid search are shown below in Figure 2, with the best parameters being learning rate 0.001 and dropout 0.3. Notably, it did appear that dropout rates do not affect the results, suggesting that weight decay had already provided enough regularisation. The Figure 3 shows the training and the validation loss curves for the best configuration. The learning curve ran up to 24 epochs, with both curves remaining close together, demonstrating good generalisation. The small gap between the two did imply that minimal overfitting is taking

```
Early stopping at epoch 24
lr=0.001, dropout=0.3: Train Loss=0.6411, Val Loss=0.6514, Time=33.0s
Early stopping at epoch 33
lr=0.001, dropout=0.5: Train Loss=0.6542, Val Loss=0.6517, Time=41.9s
Early stopping at epoch 19
lr=0.01, dropout=0.3: Train Loss=0.6885, Val Loss=0.6732, Time=44.3s
Early stopping at epoch 10
lr=0.01, dropout=0.5: Train Loss=0.7121, Val Loss=0.6778, Time=16.0s
Early stopping at epoch 14
lr=0.1, dropout=0.3: Train Loss=0.8518, Val Loss=0.8006, Time=45.7s
Early stopping at epoch 14
lr=0.1, dropout=0.5: Train Loss=1.6425, Val Loss=0.8006, Time=45.6s

Best parameters: {'lr': 0.001, 'dropout': 0.3}
```

Figure 2

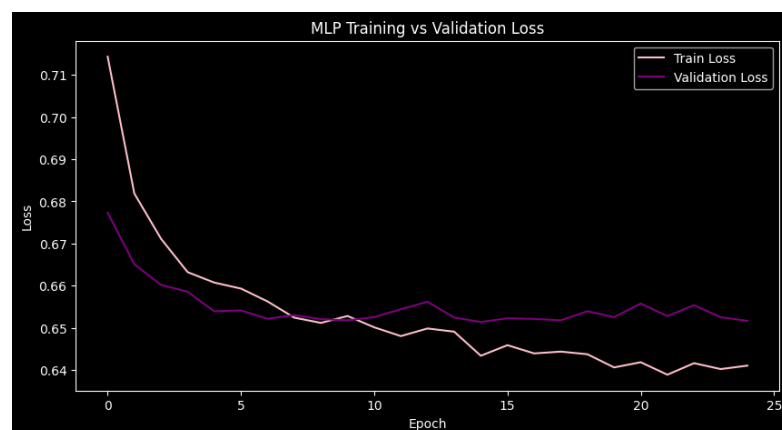


Figure 3

place. While a third hidden layer architecture was tested, it did not produce any improvements, and was therefore not included in the final model.

The best MLP model achieved a ROC-AUC of 0.777 and F1 score 0.54. The disparity between the two suggest that while the model can discriminate across all thresholds, performance at the standard 0.5 decision boundary is suboptimal for imbalanced data. The training time for the best model was 33 seconds, allowing good computation time.

The second method, SVMs, utilised RBF kernels. The choice was made after careful consideration of the type of data being analysed. Credit card default behaviour is unlikely to be linearly separable in a 23-feature dimensional space. RBF kernels address this by mapping data in infinite dimensional space, allowing it to capture non-linear boundaries between the two classes (Cortes and Vapnik, 1995). While ideally, the grid search was going to be completed on the full training set, this appeared to be computationally infeasible. Therefore, the search occurred on a stratified subset of 5000 samples, which significantly reduced computation time. The best parameters found were  $C=10$  and  $\gamma=0.01$ ; the former allowing for fewer margin violations, whilst the latter providing a moderate kernel width, which created a balance between bias and variance.

The final SVM model was retrained on the full training set to maximise its performance, completing in 83 seconds. The introduction of a polynomial kernel ( $\text{degree}=2$ ,  $C=1$ ,  $\text{coef0}=1$ ) was also attempted as an alternative, however, its results were marginally worse, confirming RBF as the correct kernel choice. In addition, the use of `class_weight` automatically adjusted the weight of the two classes, as inversely proportional to their class frequency. This ensured that both classes were equally accounted for in the training process.

The results of the SVM model demonstrated this method's ability to the classes (58%), however all other results were below and/or the same as the MLP. The SVMs' ROC-AUC curve reached 0.760 and the F1 of 0.53 or mildly below the MLP's.

### Models Comparison:

While measuring accuracy levels for an imbalanced dataset can be counterproductive, as mentioned above, the MLP's 80% and SVM's 77% accuracy did reflect overall improvement in classifying both outcomes. The similar ROC-AUC results (0.78 and 0.75), in addition, suggest strong discriminative abilities across all thresholds, particularly in the MLP model.

The precision score was marginally better in the MLP model. This does demonstrate higher abilities at correctly recognising the two classes (56% as opposed to the SVM's 49%), nonetheless, the average F1 score (54%), showcases the model's inability to correctly classify instances, regardless of its superior precision score.

Interestingly, the SVM correctly identified 58% of actual defaults, as opposed to the MLP's 51%. This does emphasize an important distinction between the two models: whilst the MLP appears to have provided the overall superior results, the SVM is more able to capture the

```
MLP Results:
      precision    recall  f1-score   support

     0       0.87       0.88       0.87       3504
     1       0.56       0.52       0.54        996

 accuracy                   0.80       4500
 macro avg       0.71       0.70       0.71       4500
 weighted avg    0.80       0.80       0.80       4500

ROC-AUC: 0.777544825420403
Confusion Matrix:
[[3092  412]
 [ 479  517]]
```

```
SVM Results:
      precision    recall  f1-score   support

     0       0.87       0.83       0.85       3504
     1       0.49       0.58       0.53        996

 accuracy                   0.77       4500
 macro avg       0.68       0.70       0.69       4500
 weighted avg    0.79       0.77       0.78       4500

ROC-AUC: 0.7600663785278099
Confusion Matrix:
[[2899  605]
 [ 421  575]]
```

sensitivity to the minority class- a critical metric when focusing on financial data, where missing a default can have direct consequences for the lender.

Given these results, it becomes apparent that neither one of the models completely dominates. Whilst making an informed choice when selecting methodology, it is important to keep in mind that both models have specific strengths, and the choice depends on necessity rather than overall performance alone. Therefore, if the minimisation of missing defaults is of critical importance, the SVM model becomes the preferred choice; on the other hand, if overall discriminative performance is of essence, the MLP model should be selected.

When comparing the two ROC-AUC curves (Figure 4), the MLP curve is consistently sitting above the SVM across all thresholds, until they both converge at threshold 1.0. This visually confirms the MLP's higher ROC-AUC, and both curves' significant outperformance of the random classifier diagonal.

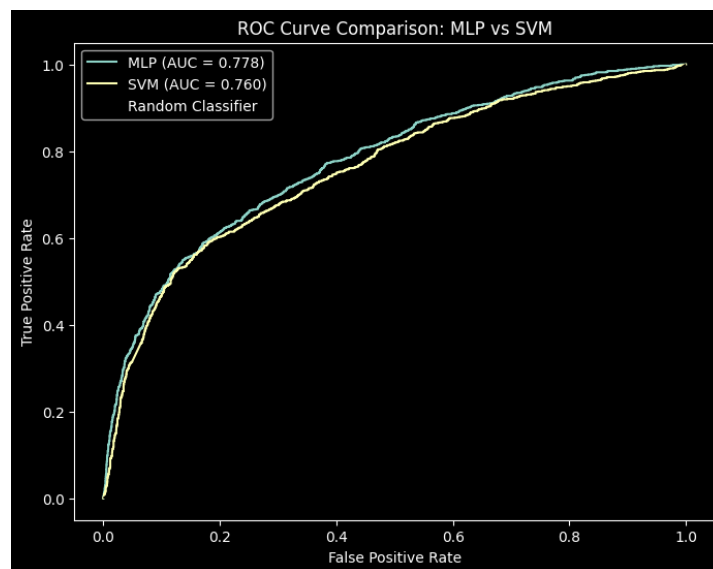


Figure 4

Additional analysis of the loss curves shows a clear divergence between the training and the validation loss occurring after epoch 20, with early stopping being triggered at epoch 24, thus saving computation. The introduction of early stopping allowed for the MLP model to prevent overfitting, and allowed the model to learn without tracing, and stopping when the divergence between training and validation has been cleared.

## Conclusion:

This assignment aimed at understanding the application of two neural computing methods when applied to the Default of Credit Card Clients dataset. This provided the perfect opportunity to apply MLP and SVM to a financial dataset, one that is both inherently noisy and complex.

Several attempts at optimising the model yielded some favourable outcomes: early stopping helped prevent overfitting, and the introduction of class\_weights and pos\_weights successfully directed the model's focus towards the minority class, creating more balanced predictions. The use of standardisation, additionally, was essential for both models, especially given the SVM's RBF kernel which is particularly distance-based.

In contrast, the attempt to introduce a third hidden layer did not produce any meaningful improvements. A similar situation occurred with attempting to use a Polynomial kernel instead

of RBF, which marginally underperformed. In addition, attempts at tuning the dropout rate and attempts with higher learning rates did not have any expected effects.

In the future, the introduction of SMOTE (Synthetic Minority Oversampling Technique) could generate synthetic default values to better balance the training data set. The creation of new ratio features could also improve the models' predictive power. Further research could also explore deeper network architectures, within which batch normalisation could be utilised.

In conclusion, while my recommendation for the model lies with the MLP, mainly due to the superior ROC-AUC and accuracy; the SVM's outperformance in recall could have better use in a financial context, which this dataset heavily depends on. My hypothesis was therefore partially confirmed, as the MLP did outperform in the majority of metrics, however the SVM's advantageous recall was an unexpected surprise, suggesting that its strengths could go beyond those of the MLP model.

## References:

- Cortes, C. and Vapnik, V. (1995) 'Support-vector networks', *Machine Learning*, 20(3), pp.273-297. doi: 10.1007/BF00994018
- Hornik, K. (1991) 'Approximation capabilities of multilayer feedforward networks', *Neural Networks*, 4(2), p.251-257, doi: 10.1016/0893-6080(91)90009-T
- Huang, C.L. *et al* (2007) 'Credit scoring with a data mining approach based on support vector machines', *Expert Systems with Applications*. 33. p.847-856. doi: 10.1016/j.eswa.2006.07.007
- Kingma, D.P. and Ba, J. (2015) 'Adam: A method for stochastic optimization', *International Conference on Learning Representations (ICLR)*. Available at: <https://www.semanticscholar.org/reader/a6cb366736791bcccc5c8639de5a8f9636bf87e8>
- Srivastava, N. *et al*. (2014) 'Dropout: A simple way to prevent neural networks from overfitting', *Journal of Machine Learning Research*, 15(1), pp.1929-1958. Available at: <https://jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf>
- Yeh, I.C. and Lien, C.H. (2009) 'The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients', *Expert Systems with Applications*, 36(2), pp.2473-2480. doi: 10.1016/j.eswa.2007.12.020